

Deepfake Defense with Antigravity: Comprehensive Implementation Details

1. Mathematical Framework & Objective

The objective of our framework is to introduce an imperceptible, adversarial "watermark" into an image (I) prior to it being shared on social media. This watermark (w) is specifically designed to disrupt the feature maps of image-to-image Deepfake models (like StarGAN) while remaining robust against the severe quantization inherent in JPEG compression.

1.1 Loss Formulation

Our overall optimization objective seeks to solve:
$$\arg\max_w \left(L_{\text{disruption}}(w) - \lambda \cdot L_{\text{distortion}}(w) \right)$$

1. Disruption ($L_{\text{disruption}}$) We maximize the Mean Squared Error (MSE) between the output of the generator on the clean image ($G(I)$) and the output on the protected image ($G(I + w)$):
$$L_{\text{disruption}} = \text{MSE}\big(G(I_{\text{inv}}), c_{\text{target}}\big), G(I_{\text{clean}}), c_{\text{target}}\big)$$
 This ensures that the StarGAN generator synthesis is fundamentally shattered, resulting in severe visual artifacts instead of a convincing fake.

2. Imperceptibility ($L_{\text{distortion}}$) To constrain the watermark, we enforce an L_{∞} bound ($w \in [-\epsilon, \epsilon]$) and minimize the visual distortion:
$$L_{\text{distortion}} = \text{MSE}\big(I_{\text{inv}}, I_{\text{clean}}\big)$$
 This replaces the flawed $1/\text{PSNR}$ penalty from earlier iterations, which unintentionally minimized the PSNR instead of maximizing it during gradient ascent.

2. DCT Frequency Masking & Band Selection

Deepfake models primarily rely on low-frequency structures (shapes, edges) to align facial landmarks, while high-frequency details (textures) are aggressively stripped away by JPEG compression (e.g., $Q = 35$).

To balance survival against JPEG compression and disruption of the GAN, we strictly inject the watermark w into the **mid-frequency bands** of the Discrete Cosine Transform (DCT) domain.

2.1 Binary Mid-Frequency Mask

In an 8×8 DCT block (indexed $u, v \in [0, 7]$), we apply a binary mask M :
$$M(u, v) = \begin{cases} 1 & \text{if } 2 \leq u + v \leq 6 \\ 0 & \text{otherwise} \end{cases}$$

- **Why exclude low frequencies ($u + v < 2$)?** Altering DC and very low-frequency AC components causes massive, blocky color shifts in the spatial domain, destroying image fidelity (PSNR).
- **Why exclude high frequencies ($u + v > 6$)?** High-frequency AC components are

quantized to zero by JPEG compression. Any watermark placed here is destroyed upon saving the image.

2.2 Channel Selection

We inject the watermark exclusively into the **Luminance (Y) channel** of the YCbCr colorspace. The human eye is less sensitive to high/mid-frequency variations in luminance compared to chrominance, and JPEG compression quantizes the Cb/Cr channels far more aggressively.

3. Scale Normalization (The ϵ Bug Fix)

The most critical breakthrough in our implementation was fixing the DCT scale mismatch.

In spatial RGB, pixel values are normalized to $[0, 1]$. However, the raw DCT coefficients computed from the shifted $Y \in [-128, 127]$ channel scale into the hundreds or thousands (e.g., DC coefficient bounds are ≈ 1024).

When applying our perturbation budget $\epsilon = 0.20$ directly to raw DCT coefficients, the watermark was mathematically microscopic (representing $< 0.1\%$ of the signal power).

The Solution:

1. We introduced `dct_scale = 1000.0`.
2. Before adding w , we normalize the coefficients: $Y_{\text{norm}} = Y / 1000.0$.
3. We add the watermark: $Y_{\text{protected}} = Y_{\text{norm}} + w$.
4. We denormalize: $Y_{\text{final}} = Y_{\text{protected}} \times 1000.0$.

This ensures the ϵ budget accurately reflects a robust perturbation in the frequency domain, causing the $L2_{\text{mask_avg}}$ to leap from 0.0001 to 0.3507 , achieving a **100% Defense Success Rate (SR_{mask})**.

4. Universal Optimization & Batch Fusion

To create a **Universal Watermark** ($w_{\text{universal}}$) that generalizes across *all* images (so we don't have to optimize per-image at deployment time), we train the watermark over a dataset of images using Projected Gradient Descent (PGD).

4.1 Intra-Batch Gradient Sign Averaging

Within a single batch of $N = 8$ images, the gradient of the loss with respect to the watermark w is computed for each image. We average the *sign* of these gradients to update w : $w_{t+1} = \text{Clip}_{[-\epsilon, \epsilon]} \left(w_t + \alpha \cdot \frac{1}{N} \sum_{i=1}^N \text{sign}(\nabla_{w_i} L_i) \right) \odot M$ *Note: The binary mid-frequency mask M is multiplied element-wise ($\odot$) to ensure gradients do not bleed into unprotected frequency bands.*

4.2 Inter-Batch Exponential Moving Average (EMA)

Across different batches, the universal watermark is updated using EMA to prevent catastrophic forgetting of previous batches: $w_{\text{universal}}^{(b)} = \beta \cdot$

$$w_{\text{universal}}^{(b-1)} + (1 - \beta) \cdot w_{\text{batch}} \quad \text{\$}$$

5. Evaluation Metrics Pipeline

The framework calculates performance across two distinct domains:

5.1 Imperceptibility (Fidelity)

Measured between the clean image (I_{clean}) and the watermarked, JPEG-compressed image (I_{inv}):

- **PSNR (Peak Signal-to-Noise Ratio):** Measures pixel-level distortion in decibels.
- **SSIM (Structural Similarity Index):** Measures perceived changes in structural information (edges, textures).
- **MSE (Mean Squared Error):** The raw squared difference between the pixels.

5.2 Disruption (Defense Success)

Measured on the output of the Deepfake model:

- **L2_mask_avg:** The mean squared L2 distance between the GAN output on the clean image (G_{clean}) and the GAN output on the protected image ($G_{\text{protected}}$).
- **SR_mask (%):** The percentage of images where L2_mask_avg exceeds a predefined threshold (0.05). A 100% score indicates total disruption across the dataset.

Summary of Hyperparameters

- **ϵ (Budget):** 0.075 (Maximum allowable perturbation in the normalized DCT space)
- **α (Step Size):** 0.01 (Learning rate for PGD)
- **T (Steps):** 20 (Number of optimization iterations per batch)
- **λ (Lambda):** 0.10 (Trade-off weight between disruption and distortion)
- **Training Batch Size:** 8 (Total 64 images over 8 batches)